Clase 13 · Programa 05: chat síncrono y validación de errores

Arquitectura Cliente-Servidor · Mtro. Carlos Alberto Román Zamitiz · jueves 8 oct 2026 · Tema 2 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

1. Revisión de la Tarea 4: sk_acceptq_is_full

Función interna del kernel de Linux que decide si la **cola de aceptación** de un socket en escucha está llena. Compara con **mayor que (>)**, no con ≥.

Cliente (backlog = 1)	Condición	Resultado
1	Cola en 0	Pasa
2	¿0 > 1? No	Entra a la cola (tamaño 1)
3	¿1 > 1? No	Entra (tamaño 2)
4	¿2 > 1? Sí	**Rechazado**

> **Regla**
>
> Con backlog N caben **N + 1** conexiones esperando. El backlog no cuenta a los ya atendidos.
>
> Comandos (como root): `sudo sysctl -w net.ipv4.tcp_syncookies=0` y `sudo sysctl -w net.ipv4.tcp_syn_retries=1`.

2. Programa 05: chat bidireccional síncrono

Par `programa_05_servidor_inet.c` / `programa_05_cliente_inet.c`: chat en texto tipo walkie-talkie (uno habla y espera respuesta).

- Usa **`PF_INET`** en vez de `AF_INET`: protocol family vs. address family, **macros equivalentes**.
- `socket(PF_INET, SOCK_STREAM, IPPROTO_TCP)`; `IPPROTO_TCP` equivale a 0.

3. Validar cada llamada contra −1

```
if ((fd_s = socket(PF_INET, SOCK_STREAM, 0)) == -1) {
    perror("Error en socket");
    exit(1);
}
```

Si `socket()` falla, no tiene caso seguir con `bind`, `listen` y `accept`: todas usan su descriptor y fallarían en cascada. `perror()` es opcional; **`exit(1)` no**.

4. gethostbyaddr sin segfault

```
info_cliente = gethostbyaddr((char *)&cliente.sin_addr,
                             sizeof(cliente.sin_addr), AF_INET);
if (info_cliente == NULL)
    printf("Cliente conectado, pero no puedo obtener su origen\n");
else
    printf("Cliente conectado desde: %s\n", info_cliente->h_name);
```

Si el DNS inverso no resuelve, devuelve **NULL**, y `NULL->h_name` es una **violación de segmento**. Es la mejora del Programa 05 sobre el 04.

5. inet_ntoa de adentro hacia afuera

`inet_ntoa(*((struct in_addr *)info_cliente->h_addr))`

1. `info_cliente->h_addr` (= `h_addr_list[0]`): `char *` a los 4 bytes de la IP.
2. **Casting** `(struct in_addr *)`: tratarlo como apuntador a la estructura de red, no como texto.
3. **Desreferencia** con el `*` de afuera: la estructura `in_addr` completa.
4. `inet_ntoa(…)`: la convierte a la cadena `"132.248.59.6"`.

Alternativa sencilla: `inet_ntoa(cliente.sin_addr)` con la estructura que llenó `accept()`.

6. El chat: fgets, búferes y memset

- Servidor: `while(1)` para atender un cliente tras otro y, dentro, `do-while` para la conversación.
- Teclado: `fgets(buff_respuesta, sizeof(buff_respuesta), stdin);` (stdin = descriptor 0; incluye el `'\n'`).
- Búferes `buff_peticion` y `buff_respuesta` de 256 bytes. Cliente: `send` → `recv`; servidor: `recv` → `send`.

> **memset ("esta sentencia es importantísima")**
>
> `memset(buff_peticion, 0, sizeof(buff_peticion));` al inicio de cada vuelta. Sin ella, tras "Hola me llamo Victor", el mensaje "Taco" solo sobrescribe los primeros bytes y aparece basura del anterior.

7. Quitar el '\n' y salir con "adiós"

- `recv` devuelve N bytes, incluido el salto de línea: `buff_peticion[N-1] = '\0';`.
- Si el cliente manda "adiós", el servidor sale de su ciclo y **manda una respuesta automática**: en un esquema síncrono el cliente está bloqueado en `recv()` y sin respuesta se quedaría colgado. Con ella, el cliente se destraba, ve su "adiós", sale y cierra limpio.

Error común: cerrar el cliente justo después de mandar "adiós", sin el último `recv()`.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Cola de listen. `sk_acceptq_is_full` usa `>`: con backlog N caben N + 1.

> **Robustez (examen)**
>
> Cada función de sockets en `if (… == -1) exit(1)`. Validar `NULL` antes de `->h_name`. `memset` al inicio de cada vuelta. `buff[N-1] = '\0'`.

Detalle	Para recordar
PF_INET	Equivale a AF_INET
inet_ntoa	Casting a `struct in_addr *`, desreferencia, conversión; o `inet_ntoa(cliente.sin_addr)`
fgets	Lee del teclado (stdin) incluyendo `'\n'`
"adiós"	El servidor contesta automáticamente para destrabar al cliente